

Pattern Recognition Decision Tree

How to map a problem statement to the right pattern in under 5 minutes. For each pattern: the signal phrases/keywords to listen for, and the underlying reason that signal implies that structure.

Step 0: First triage questions

Ask in this order:

1. **Is the input sorted, or can it be sorted?** → binary search / two pointers candidates
 2. **Is it about a contiguous run of an array/string?** → sliding window / prefix sum / Kadane's
 3. **Do I need all combinations/permutations/subsets?** → backtracking
 4. **Is it a graph/tree/grid with connections?** → BFS/DFS/Union-Find/Dijkstra family
 5. **Am I optimizing (min/max/count ways) with overlapping subproblems?** → DP
 6. **Do I need running "best so far" with insertion/removal at both ends?** → stack/queue family
-

Arrays / general

Signals: "given an array", "in-place", "without extra space" **Why:** Default container — first decide if it's sorted (binary search), needs window logic (sliding window), or pairwise (two pointers).

Hashing

Signals: "have we seen", "count occurrences", "find duplicate", "two numbers that sum to", "anagram", "frequency", "exists in $O(1)$ " **Why:** Any time you need $O(1)$ lookup of "have I seen this value/complement before," a hashmap/set trades space for time instead of nested loops ($O(n^2) \rightarrow O(n)$).

Linear Search

Signals: "unsorted array", "find first occurrence", small constraints **Why:** No ordering to exploit — must scan everything once. Usually a baseline, not the actual answer.

Bit Manipulation

Signals: "appears once / twice / odd number of times", "XOR", "subsets of a set", "without using extra memory", "binary representation", "power of two", "single number" **Why:** XOR cancels duplicates; AND/OR/shift operations encode subset/state info in $O(1)$ per operation — useful when values or state space are small and want $O(1)$ space tricks.

Sorting

Signals: "arrange", "order", "merge", "kth smallest/largest" (sometimes), "non-overlapping", "schedule" **Why:** Many problems become trivial once order is imposed — comparisons reduce from pairwise checks to a single linear pass after an $O(n \log n)$ sort.

Binary Search

Signals: "sorted array", "find the boundary", "minimum/maximum value such that...", "search in rotated array", "first/last position", "find peak", monotonic answer space ("can we do it with capacity X?") **Why:** Whenever you can ask "is mid feasible?" and the feasibility is monotonic (yes...yes...no...no), you can halve the search space each step — $O(\log n)$ instead of $O(n)$.

Prefix Sum / HashMap combo

Signals: "subarray sum equals k", "range sum query", "cumulative", "number of subarrays with..." **Why:** Precomputing running sums turns repeated range-sum queries from $O(n)$ each into $O(1)$, and combined with a hashmap of seen prefix sums, turns "count subarrays with sum k" into a single pass.

Sliding Window

Signals: "longest/shortest substring/subarray with...", "contains all characters", "at most K distinct", "maximum sum subarray of size k", "contiguous" **Why:** When the window only grows/shrinks monotonically as you scan (no need to recheck earlier elements), you avoid recomputation — $O(n)$ instead of $O(n^2)$ brute force.

Two Pointers

Signals: "sorted array, find pair", "palindrome check", "remove duplicates in place", "merge two sorted", "container with most water", "three sum" **Why:** On sorted/structured data, moving two indices toward/away from each other eliminates the need to check every pair — $O(n)$ instead of $O(n^2)$.

Backtracking

Signals: "all possible", "all combinations", "all permutations", "all subsets", "generate valid", "N-Queens", "Sudoku", "partition into" **Why:** Whenever the answer requires exploring a decision tree of choices with the ability to undo a choice (prune invalid branches early), backtracking is the only way to enumerate exhaustively without redundant work.

Recursion / Divide & Conquer

Signals: "split the problem", "merge sort", "quick sort", "closest pair of points", "tree recursion", problem naturally breaks into independent sub-problems of the same shape **Why:** If the problem can be split into independent halves whose solutions combine in less-than-quadratic time, D&C beats brute force (e.g., merge sort $O(n \log n)$ vs $O(n^2)$).

Monotonic Stack

Signals: "next greater element", "next smaller element", "daily temperatures", "largest rectangle in histogram", "stock span" **Why:** When you need, for each element, the nearest element to the left/right that breaks a trend, a stack that only keeps "still relevant" candidates lets you discard dominated elements in $O(n)$ total instead of $O(n^2)$ rescanning.

Monotonic Queue (Deque)

Signals: "sliding window maximum/minimum", "maintain max in a window as it slides" **Why:** Like a monotonic stack, but you need both ends accessible — old elements expire from the front, dominated elements get popped from the back, giving $O(1)$ amortized max-query per window slide.

Stack (general)

Signals: "matching parentheses/brackets", "undo operation", "evaluate expression", "nested structure", "DFS without recursion" **Why:** Last-in-first-out naturally models nesting/matching and lets you reverse a recursive call into an explicit loop.

Linked List

Signals: "reverse a list", "detect a cycle", "merge two lists", "remove nth node from end", "middle of list", explicitly given `ListNode` /pointers **Why:** When elements only need sequential access and frequent insert/delete (not random access), pointer manipulation avoids the $O(n)$ shift cost of arrays.

Kadane's Algorithm

Signals: "maximum subarray sum", "maximum product subarray", "maximum sum of contiguous" **Why:** A 1D DP where the optimal subarray ending at i either extends the previous one or restarts — this local decision built incrementally gives global optimum in $O(n)$.

Intervals

Signals: "meetings", "merge overlapping", "insert interval", "free time", "minimum rooms required", start/end time pairs **Why:** Sorting by start (or end) time converts an $O(n^2)$ all-pairs overlap check into a single linear sweep, since overlaps can only happen between adjacent sorted intervals.

Math & Number Theory

Signals: "GCD/LCM", "prime", "modulo", "factorial", "divisors", "power", large number constraints with "%MOD" **Why:** These problems hinge on number-theoretic properties (divisibility, primality) that have closed-form or near- $O(\log n)$ algorithms — brute force checking is usually too slow for the given constraints.

Pigeonhole Principle

Signals: " $n+1$ numbers in range $[1, n]$ ", "find the duplicate without extra space", "must exist a repeated/missing value" **Why:** If you have more items than available "slots," a collision/duplicate is guaranteed — this justifies cycle-detection or counting approaches instead of searching for proof that a duplicate exists.

Sweep Line

Signals: "overlapping intervals count", "skyline problem", "maximum overlap at any point", "events over time" **Why:** Converting start/end events into $+1/-1$ deltas and sorting by time lets you track "how many are active right now" in one linear pass instead of checking every

point against every interval.

Heap / Priority Queue

Signals: "kth largest/smallest", "top K", "median of stream", "merge K sorted lists", "task scheduler", "always get current minimum/maximum efficiently" **Why:** When you repeatedly need the current min/max while the set changes, a heap gives $O(\log n)$ insert/extract instead of $O(n)$ re-scan or $O(n \log n)$ re-sort every time.

BFS

Signals: "shortest path in unweighted graph", "minimum number of steps/moves", "level by level", "nearest", "word ladder" **Why:** BFS explores in increasing distance order, so the first time you reach the target is guaranteed to be via the shortest path (when all edges have equal weight).

DFS

Signals: "all paths", "connected components", "explore as far as possible", "is there a path", "flood fill", "maze" **Why:** When you need to fully explore a branch before backtracking (existence checks, counting components, exhaustive path search), DFS's stack-based traversal naturally matches the recursive structure of the problem.

Disjoint Set / Union-Find

Signals: "connected components", "are these two nodes connected", "number of provinces/islands (via union)", "redundant connection", "accounts merge", incremental edge additions **Why:** When connectivity queries and unions happen repeatedly and you need near- $O(1)$ "are these in the same group" checks, Union-Find (with path compression) beats re-running BFS/DFS every time.

Kahn's Algorithm (Topological Sort, BFS-based)

Signals: "course schedule / prerequisites", "build order", "task dependencies", "detect cycle in directed graph" **Why:** Repeatedly removing nodes with zero incoming edges naturally produces a valid ordering respecting dependencies, and if nodes remain stuck (nonzero indegree), that proves a cycle exists.

Tree Traversal

Signals: "binary tree", "inorder/preorder/postorder", "level order", "print tree", "serialize/deserialize" **Why:** The traversal order you choose encodes what information you need first (e.g., inorder gives sorted order for a BST; level order gives breadth-first structure).

Divide & Conquer (tree-specific)

Signals: "height of tree", "diameter", "balanced tree check", combine left/right subtree results **Why:** Tree problems are naturally recursive (a tree is two subtrees plus a root) — solving each subtree independently and combining at the root avoids re-traversing shared nodes.

Binary Search Tree (BST)

Signals: "validate BST", "kth smallest in BST", "insert/delete maintaining order", "closest value in BST" **Why:** BST's left < root < right invariant lets you prune half the tree at every node, similar to binary search, giving $O(\log n)$ average operations.

Greedy

Signals: "minimum number of", "maximum number of non-overlapping", "earliest/latest", "always pick the best available", local optimal choice claim provable **Why:** When you can prove a locally optimal choice never prevents a globally optimal solution (exchange argument), greedy avoids the exponential search space of trying all choices.

Graphs: Dijkstra's Algorithm

Signals: "shortest path with weighted edges (non-negative)", "minimum cost to reach", "cheapest flight" **Why:** Greedily expanding the currently-closest unvisited node guarantees shortest paths when weights are non-negative — a heap makes this $O(E \log V)$.

Graphs: Bellman-Ford

Signals: "shortest path with negative weights", "detect negative cycle", "at most K stops" **Why:** Dijkstra fails with negative edges because it commits too early; Bellman-Ford relaxes all edges repeatedly ($V-1$ times) to guarantee correctness even with negative weights, and one extra pass detects negative cycles.

Graphs: Floyd-Warshall

Signals: "all pairs shortest path", "small number of nodes ($\leq \sim 400$)", "find the city with smallest number of reachable cities" **Why:** When you need shortest paths between every pair (not just from one source), trying every node as an intermediate point in $O(V^3)$ is simpler and often faster overall than running Dijkstra from every node.

Graphs: Prim's / Kruskal's (MST)

Signals: "minimum cost to connect all", "minimum spanning tree", "connect all points with minimum cost" **Why:** Prim's grows a tree greedily from a start node (good for dense graphs); Kruskal's sorts all edges and adds the cheapest that doesn't form a cycle, using Union-Find (good for sparse graphs). Both exploit the cut property: the cheapest edge crossing any cut is always safe to add.

Strongly Connected Components (Kosaraju's/Tarjan's)

Signals: "mutually reachable nodes", "condense graph", "find cycles in directed graph involving groups" **Why:** In a directed graph, nodes that can reach each other form a tightly coupled unit — two DFS passes (original + reversed graph) identify these groups in $O(V+E)$.

Lowest Common Ancestor (LCA)

Signals: "common ancestor of two nodes", "lowest node that is ancestor of both" **Why:** Recursively asking "is target found in my left or right subtree" lets the first node where both sides report "found" be identified as the split point — naturally the LCA.

DP: Linear / 1D

Signals: "number of ways to climb/reach", "house robber", "maximum/minimum to reach position i depends on i-1, i-2..." **Why:** When the answer for position i can be built from a small fixed window of earlier answers, you avoid recomputing overlapping subproblems (which plain recursion would repeat exponentially).

DP: Grid-based

Signals: "unique paths in a grid", "minimum path sum", "robot moving only right/down" **Why:** Each cell's optimal answer depends only on the cell above and to the left (or similar local neighbors) — filling the grid in order reuses previously computed cells instead of re-exploring paths.

DP: Knapsack (0/1 and unbounded)

Signals: "subset sum", "partition equal subset", "coin change", "maximum value within weight limit", "each item used once / unlimited times" **Why:** Choosing to include/exclude each item under a capacity constraint has overlapping subproblems on (item index, remaining capacity) — DP table avoids recomputation that plain recursion would repeat.

DP: String (LCS / Edit Distance family)

Signals: "longest common subsequence", "edit distance", "longest palindromic substring/subsequence", comparing two strings character by character **Why:** Comparing prefixes of two strings has heavy subproblem overlap (many pairs of prefixes get compared multiple times in naive recursion) — a 2D table caches each prefix-pair result once.

DP: Interval

Signals: "matrix chain multiplication", "burst balloons", "minimum cost to merge/cut", "optimal way to split a range" **Why:** When the cost of solving a range $[i, j]$ depends on trying every split point k inside it, and overlapping sub-ranges repeat across different splits, caching by (i, j) avoids exponential recomputation.

DP: Bitmask

Signals: "traveling salesman", "visit all nodes", "assign tasks to workers", small n ($\leq \sim 20$), "subset of items used so far" **Why:** When the relevant state is "which subset of a small set has been used," encoding that subset as a bitmask integer lets you index a DP table directly — exploits the small n to make 2^n states tractable.

DP: Digit DP

Signals: "count numbers between L and R with property X ", "numbers with no repeated digits up to N ", digit constraints with very large N (up to 10^{18}) **Why:** Counting valid numbers digit-by-digit while tracking "am I still bounded by N 's prefix" (tight) avoids generating and checking every number individually — turns an $O(N)$ brute force into $O(\text{digits} \times \text{states})$.

KMP (Knuth-Morris-Pratt)

Signals: "find all occurrences of pattern in text", "string matching", repeated pattern-in-text searches needing better than $O(nm)$ **Why:** Precomputing the pattern's "longest prefix that

is also suffix" table lets you avoid re-scanning the text on a mismatch — gives $O(n+m)$ instead of naive $O(nm)$.

String Hashing / Rabin-Karp

Signals: "find pattern occurrences", "compare substrings quickly", "longest duplicate substring", "check if string is rotation of another" **Why:** Rolling hashes let you compute a substring's hash in $O(1)$ from the previous window's hash, enabling fast substring comparison without re-scanning characters each time.

Tries

Signals: "autocomplete", "prefix search", "word dictionary with wildcard", "longest common prefix among many strings" **Why:** When many strings share prefixes and you repeatedly query "does this prefix exist," a trie shares storage across common prefixes and answers prefix queries in $O(\text{length})$ regardless of dictionary size.

Binary Indexed Tree / Fenwick Tree

Signals: "range sum query with updates", "count of smaller elements after self" (with coordinate compression), frequent point updates + prefix sum queries **Why:** When you need both fast updates and fast prefix-sum queries on an array that changes, a Fenwick tree gives $O(\log n)$ for both, versus $O(n)$ for one of them with a plain array or prefix sum array.

Segment Tree

Signals: "range minimum/maximum/sum query with updates", "range update queries", more complex range queries than Fenwick can handle (min/max/gcd, lazy propagation) **Why:** Segment trees generalize Fenwick trees to any associative range operation (not just sum) and support range updates via lazy propagation — use when the query type isn't a simple prefix sum.

Bonus signal: Read the constraints first

Before even parsing the story, check N (input size) and time limit — they narrow the pattern space before you've fully understood the problem.

Constraint	What's likely allowed	Why

$N \leq 20$	Brute force, backtracking, bitmask DP	2^N or $N!$ is still small enough to enumerate exhaustively
$N \leq 100$	$O(N^3)$ triple loops, small DP tables	Cubic time is fine; this is the classic interval/grid DP zone
$N \leq 1,000$	$O(N^2)$: DP tables, BFS/DFS over all pairs	Quadratic is safe; compare-all-pairs approaches survive
$N \leq 10^5$	$O(N \log N)$ or $O(N)$: sorting, heaps, hashmaps, prefix sums	Anything quadratic times out; needs a smarter structure
$N \leq 10^6$	$O(N)$ single pass only	Even $O(N \log N)$ gets tight; one-pass scans, counting arrays
Sorted input	Binary search, two pointers	Order lets you discard half the space or move pointers monotonically instead of rechecking
Many repeated range queries	Prefix sum (static), Fenwick/Segment tree (with updates)	Precomputing avoids recomputing the same range sum/min/max every query
Tree with N nodes	DFS/recursion on subtrees, tree DP	Parent-child structure makes recursion the natural fit
Weighted graph	Dijkstra (non-negative) / Bellman-Ford (negative)	Plain BFS only works when all edges cost the same
Unweighted graph, shortest path	BFS	Level-by-level expansion guarantees shortest path first
Need top-K repeatedly	Heap	$O(\log N)$ insert/extract beats re- sorting every time
Need count/frequency	Hashmap	$O(1)$ lookup beats rescanning for duplicates
Need all possibilities	Backtracking	Only way to enumerate every valid combination/permutation/subset
Need min/max over choices, overlapping subproblems	DP	Memoizing repeated subproblems turns exponential recursion into polynomial time

Decision flow from real interview prep cheat sheets

A consolidated version of the most common “spot it when” triggers, grouped by input type — useful as a fast pre-check before diving into a problem:

Array/string input:

- Is the array sorted? → Binary Search, Two Pointers, or Prefix Sums
- Optimization (max/min/subarray)? → Sliding Window, DP, or Greedy
- Duplicates/counts/frequencies? → HashMap, HashSet, or counting array
- Substrings or fixed-size subarrays? → Sliding Window with two pointers
- Frequent min/max in a moving window? → Monotonic Queue/Deque or Heap
- Generating subsets/permutations/combinations? → Backtracking
- Matching/parsing characters (brackets, expressions)? → Stack

Graph input:

- Shortest path, unweighted? → BFS
- Shortest path, weighted? → Dijkstra, Bellman-Ford, or A*
- Connected components / cycle detection? → DFS or Union-Find
- Topological ordering (dependencies)? → Kahn’s Algorithm or DFS + visited set
- Optimization like MST? → Kruskal’s or Prim’s

Tree input (often binary trees):

- Traversals? → Inorder/Preorder/Postorder or Level-order (BFS)
- Balanced checks or diameter? → Postorder + height calculations
- Lowest Common Ancestor? → Recursive DFS or parent map + ancestor set

Linked list input:

- Detecting cycles? → Slow and Fast pointers (Floyd’s Algorithm)
- Reversals/partial changes? → Pointer juggling (prev, curr, next)
- Intersection or middle node? → Two Pointers

Curated practice sets (LeetCode numbers by pattern)

A focused 6-problem set per pattern is enough to internalize the template — more than that is diminishing returns until you hit the backlog-review stage.

#	Pattern	Problems
1	Sliding Window	3, 76, 209, 424, 567, 904
2	Two Pointers	11, 15, 16, 18, 42, 167
3	Fast/Slow Pointers (Linked List)	141, 142, 19, 876, 160, 234
4	Binary Search on Sorted Data	33, 34, 35, 153, 162, 704
5	Binary Search on Answer	875, 1011, 410, 774, 1283, 1482
6	Hashing / Frequency Maps	1, 49, 128, 217, 242, 347
7	Prefix Sum / Running Sum	303, 560, 724, 930, 974, 523
8	Difference Array / Range Updates	370, 1094, 1109, 1893, 1943, 2381
9	Monotonic Stack	739, 496, 503, 84, 85, 901
10	Monotonic Queue / Deque	239, 862, 1425, 1438, 1499, 1696
11	Heap / Top K	215, 347, 692, 703, 973, 1046
12	Intervals	56, 57, 252, 253, 435, 452
13	Greedy Scheduling / Sorting	45, 55, 406, 621, 763, 134
14	Linked List Manipulation	21, 23, 24, 25, 92, 138
15	Tree DFS	104, 112, 113, 543, 124, 226
16	Tree BFS / Level Order	102, 103, 199, 515, 637, 116
17	BST Problems	98, 99, 230, 235, 450, 700
18	Backtracking Basics	46, 47, 77, 78, 90, 39
19	Backtracking with Constraints	40, 17, 79, 131, 51, 52
20	Graph BFS / DFS	200, 695, 733, 994, 1091, 1254
21	Topological Sort / DAG	207, 210, 802, 1462, 1203, 2115

22	Union Find / DSU	547, 684, 1319, 1579, 990, 1202
23	Shortest Path	743, 787, 1514, 1631, 1334, 1976
24	MST / Graph Greedy	1584, 1135, 1168, 1489, 778, 1102
25	Trie	208, 211, 212, 648, 677, 1268
26	Bit Manipulation	136, 137, 191, 338, 268, 190
27	1D DP Basics	70, 198, 213, 322, 279, 300
28	Knapsack / Subset DP	416, 494, 518, 474, 1049, 879
29	Grid DP	62, 63, 64, 221, 931, 120
30	String DP / Sequence DP	1143, 72, 115, 583, 97, 1312

A note on memorization vs. recognition

A common trap in these prep frameworks is collecting more and more pattern lists without testing whether you can recognize the trigger live. For each pattern above, you should be able to answer four things on the spot: what problem it solves, when to use it, when *not* to use it, and the tradeoff it introduces (e.g., a hashmap trades memory for $O(1)$ lookup; a heap trades insertion simplicity for $O(\log n)$ operations instead of $O(1)$). If you can't answer those four for a pattern, more practice problems on it won't help as much as revisiting the "why" — that's the actual gap interviewers are scoring (problem understanding and approach selection, not just final code).

How to actually practice the problem sets (not just collect them)

From the same prep material — the methodology matters more than the list itself:

- These are LeetCode problem numbers, but don't grind 30 patterns at once. Work 3 patterns at a time.
- For each pattern: solve the first 2 problems to understand the idea, the next 2 for repetition, and the last 2 to stretch yourself into variants.
- After every pattern, write one reusable template from memory (not copy-pasted) — this is the actual test of whether you internalized it.
- Don't just solve and move on. Ask: what signal in the question pointed to this pattern?

That question is the whole point of the recognition exercise.

- If you get stuck, revisit the same pattern 3-4 days later. Recognition is built by spaced repetition, not cramming in one sitting.
- If you only have time to start with 10: Sliding Window, Two Pointers, Binary Search, Hashing, Prefix Sum, Monotonic Stack, Heap, Intervals, Backtracking, Graph BFS/DFS — these cover the large majority of array/string/graph interview questions.

Trigger-confidence ratings (how reliable each signal is)

Some signals are near-certain; others need a second confirming clue. Rough confidence out of 10, useful for knowing when to commit immediately vs. read one more line of the problem:

- Sliding window (substring/subarray, longest/shortest, contiguous, at most K): 10/10
- DFS/BFS on trees or graphs (shortest path, connected components, traversal): 10/10
- Two pointers (sorted array, pair sum, opposite ends): 9/10
- Binary search (sorted input, search space, monotonic answer): 9/10
- Hashmap/counting (frequency, duplicates, grouping): 9/10
- Monotonic stack/queue (next greater/smaller, sliding window min/max): 8/10

When two patterns look similar, check the secondary clue: sliding window vs. prefix sum — window works when the valid range is dynamic and contiguous, prefix sum works better for cumulative totals across many ranges; two pointers vs. sorting — two pointers cuts the search space in half on already-sorted input, sorting is the step that gets you there if it isn't sorted yet; DFS vs. BFS — DFS suits deep exploration and "does a path exist," BFS suits shortest path / fewest steps on unweighted graphs.

A decode-the-input front door (before pattern matching)

A useful first move on any new problem, before trying to map it to a pattern: label what kind of input you're holding — array/string, linked list, tree, graph/grid, intervals, "top K," "generate all," or "minimize/maximize cost." That single label cuts the candidate pattern list from ~46 down to 4-6 before you've written a line of code, because each input type has a small set of patterns that actually apply to it (e.g., "tree with N nodes" → DFS/BFS/tree DP, never sliding window).

Interview frequency context (why this investment pays off)

Across Meta, Google, Amazon, and Microsoft, the large majority of SWE interview rounds combine DSA with system design (roughly 70-85% depending on company), so the pattern recognition speed described above isn't just an academic exercise — it directly determines how many of your 45-minute rounds you clear. Common high-frequency questions worth having pre-built mental templates for: LRU/LFU cache, median from a data stream, word ladder (shortest transformation), merge K sorted lists, cycle detection in a directed graph, maximum subarray sum (Kadane + variations), Kth largest element in a stream, and designing a task scheduler with priorities — these map directly to the Heap, Hashmap+Linked List, BFS, Heap, Graph DFS/Kahn's, Kadane's, Heap, and Greedy/Heap patterns above, respectively.

Quick-reference cheat table

Keyword spotted	Likely pattern
"subarray/substring", "longest/shortest with condition"	Sliding Window
"sorted array", "find boundary/peak"	Binary Search
"pair sums to target", sorted input	Two Pointers
"all combinations/subsets/permutations"	Backtracking
"next greater/smaller element"	Monotonic Stack
"sliding window max/min"	Monotonic Deque
"shortest path, unweighted"	BFS
"shortest path, weighted non-negative"	Dijkstra
"shortest path, negative weights"	Bellman-Ford
"all pairs shortest path"	Floyd-Warshall
"connect all with min cost"	Prim's / Kruskal's
"course prerequisites / build order"	Kahn's Topological Sort
"are these connected / merge groups"	Union-Find
"number of ways to reach/climb"	Linear DP
"0/1 choice with weight & value limit"	Knapsack DP

"compare two strings, edit/match"	String DP (LCS/Edit Distance)
"count numbers with digit property"	Digit DP
"small n, visit all / subset state"	Bitmask DP
"top K / Kth largest"	Heap
"range sum + updates"	Fenwick / Segment Tree
"find pattern occurrences in text"	KMP / Rabin-Karp
"prefix search / autocomplete"	Trie
"appears once, XOR, subsets via bits"	Bit Manipulation
"n+1 values in range 1..n"	Pigeonhole
"merge overlapping meetings/intervals"	Intervals + Sorting
"max overlap at any time"	Sweep Line